

Московский Государственный Университет имени
М.В.Ломоносова

Филиал Московского Государственного
Университета имени М.В.Ломоносова в городе
Душанбе

Естественнонаучный факультет

Направление 01.03.02 "Прикладная математика и информатика"

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

**«ОБНАРУЖЕНИЕ СЕЧЕНИЙ, СООТВЕТСТВУЮЩИХ
ЗАДАННОМУ ОБРАЗЦУ»**

Выполнил:

студент 4 курса направления
01.03.02 "Прикладная математика и информатика"
очной формы обучения
Парпиев Осимхон Сухробович

Научный руководитель:

доцент кафедры вычислительной математики
механико-математического факультета МГУ
Валединский Владимир Дмитриевич

Душанбе - 2020

ОБНАРУЖЕНИЕ СЕЧЕНИЙ,
СООТВЕТСТВУЮЩИХ ЗАДАННОМУ
ОБРАЗЦУ

Парпиев Осимхон Сухробович

24 июня 2020 г.

Оглавление

1	Введение	3
1.1	Происхождение задачи	3
1.2	Общий подход к решению задачи	5
1.2.1	Основные понятия	5
1.2.2	Постановка задачи	7
1.2.3	Декомпозиция задачи	7
2	Практическая часть	9
2.1	Классы	9
2.1.1	Точки и векторы на R_2 "Point2D"	9
2.1.2	Точки и векторы на R_3 "Point3D"	10
2.1.3	Прямые на R_2 "Line2D"	10
2.1.4	Прямые на R_3 "Line3D"	11
2.1.5	Отрезки и рёбра на R_2 "Edge2D"	11
2.1.6	Отрезки и рёбра на R_3 "Edge3D"	11
2.1.7	Плоскость "Plane"	11
2.1.8	Грань многогранника "Face"	12
2.1.9	Многогранник "Polyhedron"	12
2.1.10	Многоугольник на R_2 "Polygon2D"	12
2.1.11	Многоугольник на R_3 "Polygon3D"	12
3	Алгоритмы	13
3.1	Построение сечения	13
3.1.1	Алгоритм	13
3.1.2	Псевдокод алгоритма	15
3.2	Нахождение центра тяжести многоугольника в R_2	18
3.2.1	Алгоритм	18
3.2.2	Псевдокод алгоритма	18
3.3	Нахождение габаритов многоугольника	19
3.3.1	Алгоритм	19
3.3.2	Псевдокод алгоритма	19

3.4	Нахождение площади многоугольника	20
3.4.1	Алгоритм	20
3.4.2	Псевдокод алгоритма	20
3.5	Нахождение сечения заданной площади, заданным направ- лением сечения	21
3.5.1	Алгоритм	21
3.5.2	Псевдокод алгоритма	21
3.6	Вычисление нормы разности многоугольников	24
3.6.1	Алгоритм	24
3.6.2	Псевдокод алгоритма	26
3.7	Аналитическое вычисление глубины сечения	28
3.8	Поиск направления сечения шагами по двумерной сетке . .	30
3.8.1	Алгоритм	30
3.8.2	Псевдокод алгоритма	30
3.9	Аналитический метод решения задачи (не реализован) . . .	34
4	Заключение	36
4.1	Выводы	36
4.2	Результаты	37
	Список алгоритмов	46
	Список литературы	47

Глава 1

Введение

1.1 Происхождение задачи

Данная работа возникла из вопроса автоматизации производственного процесса огранки драгоценных камней. Огранка подразумевает создание на поверхности камня ряда геометрически правильных плоскостей, граней, от которых будут отражаться попадающий внутрь кристалла свет. После многократного внутреннего отражения и преломления лучи света разделяются на спектральные составляющие и, покидая камень, создают игру оттенков на его поверхности.[1]

Качественно выполненная огранка существенно повышает стоимость драгоценного камня, в то же время неправильная огранка может «убить» камень, внести в него дефекты.

Процесс огранки большого количества драгоценных камней в промышленных масштабах требует как можно большего упрощения и автоматизации, поэтому перед тем, как начать манипуляции с драгоценным камнем, требуется построить его модель, работая с которой, можно задать алгоритм работы автоматизированного станка для шлифовки камня.

Одна из центральных задач – построение трехмерных моделей физических тел (камней) в процессе их огранки. С точки зрения математических постановок, обработка может заключаться в поиске отснятых граней на поверхности отшлифованного камня или в поиске сечений текущей трехмерной модели так, чтобы форма сечения соответствовала отснятому контуру реальной грани.

В данной дипломной работе рассматривается частный аспект подобной обработки – обнаружение сечений, соответствующих заданному образцу. Например, есть план огранки полуфабриката «Груша».

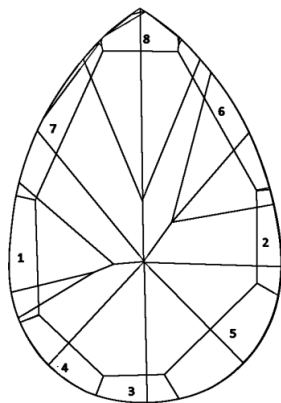
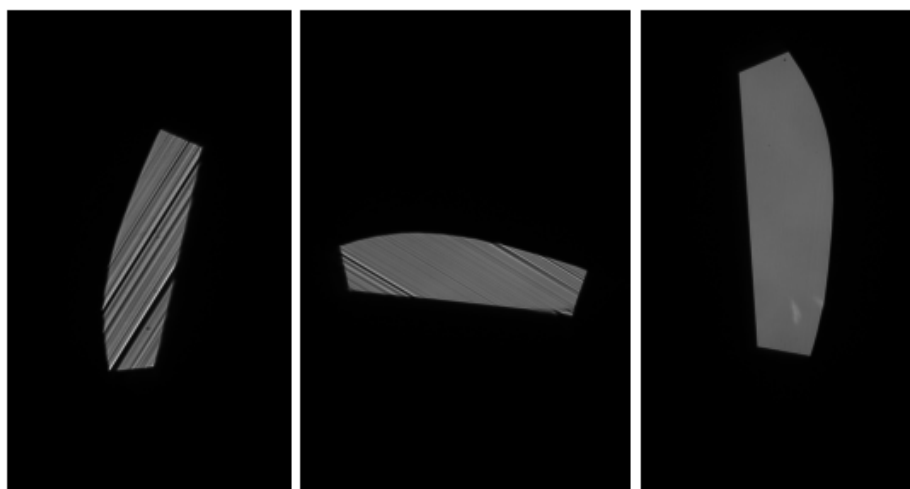


Рис: План огранки "Груша"

Для реализации огранки «Груша», нужно определять сечения модели, которые будут давать такую же картинку грани.



1.2 Общий подход к решению задачи

1.2.1 Основные понятия

Определение. *Пространственным многоугольником* называется тройка $I = (P, F, R)$, где P - множество вершин, F - множество граней, $R \subseteq P \times F$ - множество рёбер: пара $(p, f) \in R$, если вершина p должна лежать в грани f . [5]

Определение. *Гранью* называется тройка (i, π_i, L_i) , где i - номер грани, π_i - плоскость $A_i x + B_i y + C_i z + D = 0$, в которой лежит грань, L_i - список номеров вершин, которые содержатся в грани. [4]

Многогранник задается так:

Во-первых, задается количество вершин и количество граней многогранника. Во-вторых, перечисляются координаты всех вершин (это тройки чисел x, y, z). В-третьих перечисляются грани.

Определение. *Плоским многоугольником* называется пара (V, E) , где V - множество вершин, которые лежат на одной плоскости, и первый и последний элемент этого множества совпадают, $E \subseteq V \times V$ - множество рёбер (сторон): пара $(v_i, v_j) \in E$, если v_i и v_j являются соседними.

Определение. *Соседними вершинами* называются вершины многоугольника, которые являются концами одной из его сторон.

Определение. *Выпуклый многоугольник* - многоугольник, все точки которого лежат по одну сторону от любой прямой, проходящей через две его соседние вершины. [5]

В данной работе будем рассматривать только выпуклые многоугольники.

Многоугольник задается либо через упорядоченный список вершин $((v_i, v_{i+1})$ - соседние), либо через список сторон (рёбер).

Рассмотрим полярную систему координат, на которой каждая точка задается координатами r - полярный радиус и φ - полярный угол. Предположим, что центр тяжести выпуклого многоугольника совпадает с началом рассматриваемой системы координат, тогда многоугольник может быть задан как некоторая 2π - периодическая функция $r = r(\varphi), \varphi \in$

$[0, 2\pi)$ в полярных координатах.

С целью дискретизации задачи, рассмотрим пространственную область в виде отрезка $\Omega = [0, 2\pi]$, где вводится равномерная сетка с шагом $h : \{x_j = jh \mid j = 0, \dots, N\}$, где $Nh = 2\pi$. Тогда вычисленные значения r_j , $j = 0, \dots, N-1$, функции $r = r(\varphi)$, $\varphi \in [0, 2\pi)$ в узлах сетки ω_h задает дискретное представление многоугольника в полярной плоскости. Таким образом, такое представление многоугольника позволяет свести задачу к анализу дискретных периодических функций на полярных сетках.

Пусть заданы два многоугольника как 2π - периодические функции r и g в полярных координатах. Значение функций r и g , определенных на сетке ω_h , в узле j обозначим r_j и g_j соответственно. Теперь определим норму разности этих многоугольников:

Определение. *Нормой разности плоских многоугольников или расстоянием между плоскими многоугольниками называют величину:*

$$\| r - g \| = \sqrt{\frac{1}{N} \sum_{j=1}^N (r_j - g_j)^2}$$

Сдвигом (или поворотом) функции r на сетке ω_h назовем такую зависимость от целочисленного параметра s , что $r_i \mapsto r_{i+s}$. Причем:

если $i + s > N$, то $r_{i+s} \mapsto r_{i+s-N}$

Иными словами, значения r продолжаются по N -периодичности. Число s назовем величиной сдвига (поворота) функции r . Сеточную функцию r , к которой применили поворот величины s , будем обозначать r^s . Тогда:

$$\| r^s - g \| = \sqrt{\frac{1}{N} \sum_{j=1}^N (r_{j+s} - g_j)^2}$$

1.2.2 Постановка задачи

Дана модель трёхмерного тела (выпуклый многогранник), дан плоский многоугольник (образец) и дано начальное направление поиска сечения (начальное приближение к нормали грани сечения). Требуется найти сечение, наиболее близкое по форме к заданному образцу, и определить погрешность совпадения сечения и многоугольника - образца.

1.2.3 Декомпозиция задачи

Эту задачу предлагается решать итерационным методом, где на каждом шаге выбирается некоторое сечение и это сечение сравнивается с заданным многоугольником. Задача распадается на пять этапов:

- 1 Построение сечения
- 2 Поиск наилучшего совмещения двух плоских многоугольников
- 3 Поиск сечения заданной площади с заданной нормалью
- 4 Поиск направления для сечения с наилучшим совпадением с образцом.
- 5 Вычисление расстояния между найденной гранью сечения и заданным образцом

Этап 1 состоит из следующих шагов:

- Нахождение пересечения ребра и плоскости
- Нахождение пересечения многогранника и плоскости в виде плоского многоугольника в пространстве
- Перевод плоского многоугольника из пространства на плоскость

Этап 2 состоит из следующих шагов:

- Нахождение центров тяжести плоских многоугольников
- Перенос многоугольника на полярную сетку
- Поиск поворота наилучшего совмещения

Один из совмещаемых многоугольников является «обычной» геометрической фигурой – многоугольником с относительно небольшим числом сторон, а другой многоугольник представлен «пиксельным контуром», полученным с цифровой фотографии, и может интерпретироваться как многоугольник с значительным количеством вершин (несколько тысяч).

Этап 3 состоит из следующих шагов:

- Нахождение примерной середины многогранника
- Нахождение сечения с максимальной площадью
- Нахождение сечения с минимальной площадью
- Нахождение сечения заданной площади методом деления отрезка пополам

Этап 4 состоит из следующих шагов:

- Нахождение нормы разности текущего сечения и образца
- Нахождение сечения с наилучшим совпадением с образцом в смысле максимального диаметра путём наклона нормали плоскости сечения шагами
- Нахождение сечения с наилучшим совпадением с образцом в смысле габаритов путём наклона нормали плоскости сечения шагами
- Нахождение сечения с наилучшим совпадением с образцом в смысле сеточной нормы путём наклона нормали плоскости сечения шагами

Этап 5 состоит из следующих шагов:

- Поиск наилучшего совмещения сечения и образца
- Вычисление сеточной нормы разности сечения и образца

Для реализации описанных этапов была разработана система геометрических классов для представления объектов и составлены алгоритмы для реализации отдельных шагов. Эти классы и алгоритмы описаны в следующих разделах работы.

Глава 2

Практическая часть

2.1 Классы

2.1.1 Точки и векторы на R_2 "Point2D"

Свойства класса

1: x, y ▷ Вещественные числа

Методы класса

1: $a.\text{DISTANCE}(b)$ ▷ Вычисляет расстояние от a до b

$$d = \sqrt{(a_x - b_x)^2 + (a_y - b_y)^2}$$

2: $V.\text{LENGTH}()$ ▷ Вычисляет длину вектора

$$d = \sqrt{V_x^2 + V_y^2}$$

3: $V.\text{NORMALIZE}()$ ▷ Нормализует вектор

$$d = \sqrt{V_x^2 + V_y^2}, \quad V_x \leftarrow V_x/d, \quad V_y \leftarrow V_y/d$$

4: $a.\text{ROTATE}(\varphi)$ ▷ Поворот относительно начала координат против часовой стрелки

$$\begin{pmatrix} a_x \\ a_y \end{pmatrix} \leftarrow \begin{pmatrix} \cos \varphi & -\sin \varphi \\ \sin \varphi & \cos \varphi \end{pmatrix} \times \begin{pmatrix} a_x \\ a_y \end{pmatrix}$$

2.1.2 Точки и векторы на R_3 "Point3D"

Свойства класса

1: x, y, z ▷ Вещественные числа

Методы класса

1: $a.DISTANCE(b)$ ▷ Вычисляет расстояние от a до b

$$d = \sqrt{(a_x - b_x)^2 + (a_y - b_y)^2 + (a_z - b_z)^2}$$

2: $V.LENGTH()$ ▷ Вычисляет длину вектора

$$d = \sqrt{V_x^2 + V_y^2 + V_z^2}$$

3: $V.NORMALIZE()$ ▷ Нормализует вектор

$$d = \sqrt{V_x^2 + V_y^2 + V_z^2}, V_x \leftarrow V_x/d, V_y \leftarrow V_y/d, V_z \leftarrow V_z/d$$

2.1.3 Прямые на R_2 "Line2D"

Уравнение прямой на R_2 :

$$Ax + By + C = 0$$

Свойства класса

1: A, B, C ▷ Вещественные числа

Методы класса

1: $L_1.INTERSECT(L_2)$ ▷ Возвращает точку пересечения L_1 и L_2
Координаты точки пересечения - решение системы:

$$\begin{cases} A_1x + B_1y + C_1 = 0 \\ A_2x + B_2y + C_2 = 0 \end{cases}$$

2.1.4 Прямые на R_3 "Line3D"

Каноническое уравнение прямой в пространстве:

$$\frac{x - x_0}{m} = \frac{y - y_0}{n} = \frac{z - z_0}{p}$$

(m, n, p) - координаты нормального вектора прямой

Свойства класса

1: x_0, y_0, z_0, m, n, p ▷ Вещественные числа

2.1.5 Отрезки и рёбра на R_2 "Edge2D"

Свойства класса

1: a, b ▷ Объекты класса *Point2D*
2: L ▷ Объект класса *Line2D*

2.1.6 Отрезки и рёбра на R_3 "Edge3D"

Свойства класса

1: a, b ▷ Объекты класса *Point3D*
2: L ▷ Объект класса *Line3D*

2.1.7 Плоскость "Plane"

Уравнение плоскости:

$$Ax + By + Cz + D = 0$$

Свойства класса

1: A, B, C, D ▷ Вещественные числа

Методы класса

1: $P.INCLINE_X(step)$ ▷ Наклоняет вектор нормали плоскости P на шаг $step$ влево и вправо в зависимости от знака $step$

$$P \leftarrow Ax + (B + step)y + Cz + D = 0$$

2: $P.INCLINE_Y(step)$ ▷ Наклоняет вектор нормали плоскости P на шаг $step$ вверх и вниз в зависимости от знака $step$

$$P \leftarrow Ax + By + (C + step)z + D = 0$$

2.1.8 Грань многогранника "Face"

Свойства класса

- 1: A ▷ Объект класса $Plane$
- 2: $PointNumber$ ▷ Список номеров вершин, которые лежат на грани

2.1.9 Многогранник "Polyhedron"

Свойства класса

- 1: $Points$ ▷ Вершины многогранника - список объектов класса $Point3D$
- 2: $Edges$ ▷ Рёбра многогранника - список объектов класса $Edge3D$
- 3: $Faces$ ▷ Грани многогранника - список объектов класса $Face$

2.1.10 Многоугольник на R_2 "Polygon2D"

Свойства класса

- 1: $Sides$ ▷ Рёбра многоугольника - список объектов класса $Edge2D$
- 2: $Points$ ▷ Вершины - список объектов класса $Point2D$

Методы класса

- 1: $P.GETAREA()$ ▷ Вычисляет площадь P . См. главу 3.4
- 2: $P.ROTATE(\varphi)$ ▷ Поворачивает P относительно начала координат

$$P.Points_i \leftarrow P.Points_i.ROTATE(\varphi), i \in \overline{1, n}$$

- 3: $P.Center$ ▷ Вычисляет центр тяжести P . См. главу 3.2
- 4: $P.SHIFTTOCENTER(Point)$ ▷ Совмещает центр тяжести P с началом координат

$$P.Points_i \leftarrow P.Points_i - P.Center, i \in \overline{1, n}$$

- 5: $P.SIZE()$ ▷ Вычисляет габариты P . См. главу 3.3

2.1.11 Многоугольник на R_3 "Polygon3D"

Свойства класса

- 1: $Sides$ ▷ Рёбра многоугольника - список объектов класса $Edge3D$
- 2: $Points$ ▷ Вершины - список объектов класса $Point3D$

Глава 3

Алгоритмы

3.1 Построение сечения

3.1.1 Алгоритм

Дан многогранник с вершинами $P_i, i \in \{1, n\}$ и с гранями $F_j, j \in \{1, k\}$. Для каждой грани заданы номера принадлежащих ей вершин. Дана плоскость сечения `SectionPlane`. При пересечении многогранника плоскостью, у нас будут три случая:

- 1 Плоскость не пересекает многогранник. Если плоскость сечения касается какой-то вершины или ребра, будем считать что нет пересечения.
- 2 Плоскость касается какой-то грани $F_l, l \in (1, k)$
- 3 Плоскость сечения пересекает многогранник

Первый случай: Нет сечения.

Второй случай: Грань, которая получается в сечении - это грань F_l .

Третий случай: В этом случае, плоскость сечения пересекает какие-то грани многогранника, а какие-то не пересекает. На месте пересечения грани с плоскостью мы получим одно ребро грани, которая получается в сечении. Найдем все такие ребра, соединим их и получим многоугольник, который получается в сечении. Полученный многоугольник - плоский и в R_3 . Для того чтобы перенести его на R_2 , заменим базис. Первым базисным вектором e_1 должен быть нормальный вектор плоскости, на которой

лежит многогранник. Второй вектор e_2 - любой ортогональный вектор первому. Третий вектор e_3 получается из векторного произведения первых двух. Нормируем и получим ортонормированный базис.

$$e_1 = (P_1 - P_2) \times (P_1 - P_3)$$

$$e_2 = \{\angle(e_1, e_2) = \frac{\pi}{2}\}$$

$$e_3 = e_1 \times e_2$$

На этом базисе многоугольник будет параллелелен плоскости $\langle e_2, e_3 \rangle$, поэтому проекция многоугольника на эту плоскость будет равна самому многоугольнику. Эта проекция будет нашим представлением многоугольника на R_2 .

Чтобы найти координаты новой точки X' в новом базисе, надо решить систему уравнений:

$$\begin{pmatrix} (e_1)_i & (e_2)_i & (e_3)_i \\ (e_1)_j & (e_2)_j & (e_3)_j \\ (e_1)_k & (e_2)_k & (e_3)_k \end{pmatrix} \times \begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

3.1.2 Псевдокод алгоритма

Пересечение ребра с плоскостью

Algorithm 1 Пересечение ребра с плоскостью

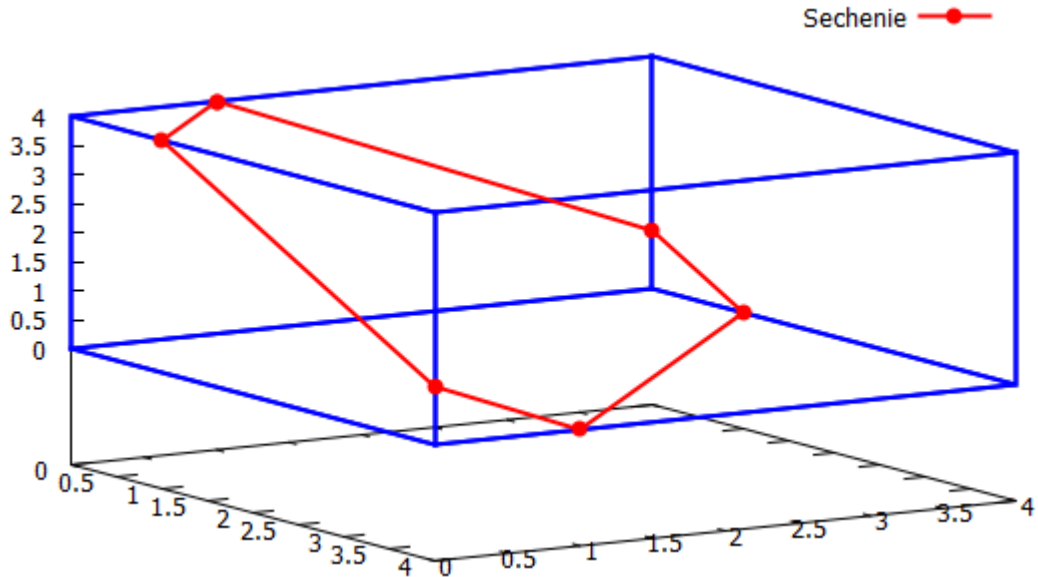
```
function INTERSECT(Edge, SectionPlane)  
  if AREINTERSECT(Edge.Line, SectionPlane) then  
    Point  $\leftarrow$  INTERSECT(Edge.Line, SectionPlane)  $\triangleright$  Находим точку  
    пересечения прямой, на которой лежит ребро, и плоскости  
  else  
    return 0  
  end if  
  if POINTINSEGMENT(Point, Edge) then  $\triangleright$  Проверяем принадлежит  
  ли точка ребру  
    return Point  
  else  
    return 0  
  end if  
end function
```

Algorithm 2 Пересечение грани и плоскости

```
function INTERSECT(Face, SectionPlane)  
  for  $i = 0$  to  $k$  do  $\triangleright k$ -количество рёбер на грани  
    if AREINTERSECT(Face.Edgei, SectionPlane) then  
      Points.Push_Back(INTERSECT(Face.Edgei, SectionPlane))  
    end if  
  end for  $\triangleright$  В списке Points ровно две точки  
  Edge(Ponts.Front, Points.Back)  
  return Edge  
end function
```

Algorithm 3 Алгоритм построения сечения в R_3

```
1: function SECTION3D(Polyhedron,SectionPlane)
2:    $m \leftarrow 0$ 
3:   for  $i = 1$  to  $k$  do
4:     if  $Polyhedron.Face_i.GetPlane() = SectionPlane$  then
5:        $Result \leftarrow Polyhedron.Face_i$ 
6:       return  $Result$ 
7:     end if
8:      $Edge_m = Intersect(Polyhedron.Face_i, SectionPlane)$ 
9:      $m \leftarrow m + 1$ 
10:  end for
11:  if  $m = 0$  then
12:    Print{Нет сечения!}
13:    return 0
14:  end if
15:  for  $i = 0$  to  $m$  do
16:     $Result.push\_back(Edge_i)$ 
17:  end for
18:  return  $Result$ 
19: end function
```



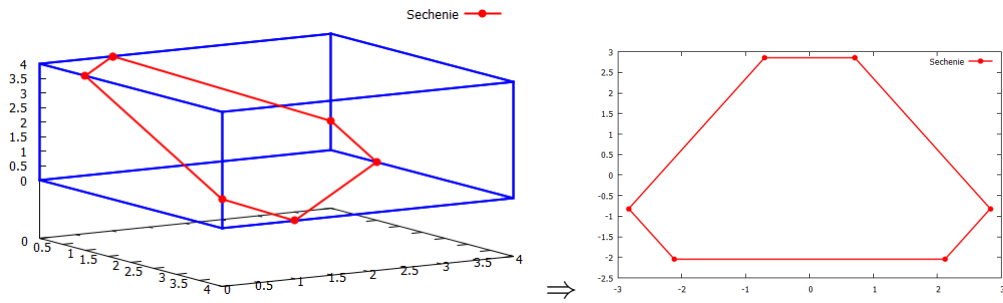
Перевод плоского многоугольника из R_3 в R_2 :

Algorithm 4 Перевод плоского многоугольника из R_3 в R_2

```

1: function GETPOLYGON2D(Polygon3D)
2:    $p_1 \leftarrow Polygon3D.Point_1$ 
3:    $p_2 \leftarrow Polygon3D.Point_2$ 
4:    $p_3 \leftarrow Polygon3D.Point_3$ 
5:    $Plane\ OXY(p_1, p_2, p_3)$ 
6:    $e_3 \leftarrow OXY.GETNORMAL( )$   $\triangleright e_3$  один из векторов нового базиса
7:    $e_2 \leftarrow ORTHOGONALVECTOR(e_3)$ 
8:    $e_1 \leftarrow VECTORPRODUCT(e_2, e_3)$ 
9:    $e_1.NORMALIZE( )$ 
10:   $e_2.NORMALIZE( )$ 
11:   $e_3.NORMALIZE( )$   $\triangleright$  Получили ортонормированный базис, в
    котором Polygon3D будет параллелен с осью OXY
12:  for  $i = 0$  to  $n$  do  $\triangleright n$  - количество вершин в многоугольнике
13:     $Result.Push\_Back(POINTINBASIS(Polygon3D.Point_i, e_1, e_2, e_3)_{xy})$ 
14:  end for
15:  return  $Result$ 
16: end function

```



Построение сечения в R_2

Algorithm 5 Построение сечения на плоскости

```

function SECTION2D(Polyhedron, SectionPlane)
   $Polygon3D \leftarrow SECTION(Polyhedron, SectionPlane)$ 
   $Result \leftarrow GETPOLYGON2D(Polygon3D)$ 
  return  $Result$ 
end function

```

3.2 Нахождение центра тяжести многоугольника в R_2

Центр тяжести треугольника - это точка пересечения медиан.

3.2.1 Алгоритм

Разобьем многоугольник на треугольники и найдём их центры тяжести. Усредним эти центры тяжести треугольников с весами, равными их площади.

3.2.2 Псевдокод алгоритма

Algorithm 6 Нахождение центра тяжести многоугольника

```
1: function CENTEROFPOLYGON(Polygon2D)
2:   for  $i = 0$  to  $n$  do            $\triangleright n$  - количество вершин в многоугольнике
3:      $c_0 \leftarrow c_0 + \text{Polygon2D.Point}_i$ 
4:   end for
5:    $c_0 \leftarrow c_0 / n$ 
6:   for  $i = 0$  to  $n$  do
7:      $p_1 \leftarrow \text{Polygon2D.Point}_i$ 
8:     if  $i + 1 \neq n$  then
9:        $p_2 \leftarrow \text{Polygon2D.Point}_{i+1}$ 
10:    else
11:       $p_2 \leftarrow \text{Polygon2D.Point}_0$ 
12:    end if
13:     $a \leftarrow p_1 - c_0$ 
14:     $b \leftarrow p_2 - c_0$ 
15:     $s \leftarrow \text{VectorProduct2D}(a, b)$             $\triangleright s$  - Площадь  $\triangle p_1 p_2 c_0$ 
16:     $sum \leftarrow sum + s$ 
17:     $c \leftarrow (p_1 + p_2 + c_0) / 3.0$             $\triangleright c$  - Центр тяжести  $\triangle p_1 p_2 c_0$ 
18:     $Center \leftarrow Center + c$ 
19:  end for
20:   $Center \leftarrow Center / sum$ 
21:  return  $Center$ 
22: end function
```

3.3 Нахождение габаритов многоугольника

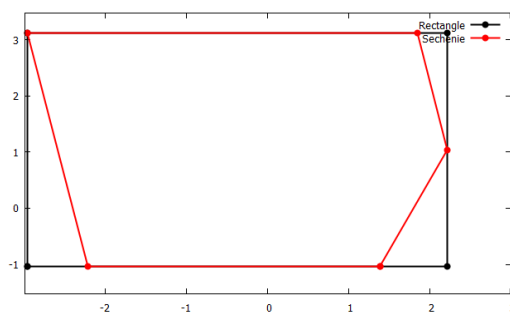
3.3.1 Алгоритм

Габариты многоугольника - это наименьший прямоугольник, который содержит в себе этот многоугольник. Для того чтобы найти этот прямоугольник, надо повернуть многоугольник так, чтобы одна сторона стала параллельной оси Ox , и вычислить длину проекций многоугольника на ось Ox и ось Oy . Эти длины проекций будут длиной и высотой прямоугольника. Найдем такие прямоугольники для всех поворотов многоугольника. Наименьший такой прямоугольник будет габаритами многоугольника.

3.3.2 Псевдокод алгоритма

Algorithm 7 Нахождение габаритов многоугольника

```
1: function SIZE(Polygon2D)
2:    $BestRectangle \leftarrow MaxRectangle$ 
3:   for  $i = 0$  to  $k$  do ▷ k-Количество рёбер многоугольника
4:      $Angle \leftarrow Angle(Polygon2D.Side_i, Ox)$ 
5:      $Polygon2D.Rotate(Angle)$  ▷ Повернули многоугольник так,
чтобы одна сторона стала параллельной оси  $Ox$ 
6:      $a \leftarrow ProjectionLength(Polygon2D, Ox)$ 
7:      $b \leftarrow ProjectionLength(Polygon2D, Oy)$  ▷  $a, b$ -Длины проекций
на ось  $Ox$  и  $Oy$ 
8:     if  $Rectangle(a, b) < BestRectangle$  then
9:        $BestRectangle \leftarrow Rectangle(a, b)$ 
10:    end if
11:  end for
12:  return  $BestRectangle$ 
13: end function
```



3.4 Нахождение площади многоугольника

3.4.1 Алгоритм

Дан многоугольник с вершинами P_i , $i = \{1, n\}$. Фиксируем точку C_0 внутри многоугольника и соединим эту точку отрезком со всеми вершинами многоугольника. Получим набор из n треугольников $t_i = \triangle C_0 P_i P_{i+1}$, $i = \{1, n-1\}$ и $t_n = \triangle C_0 P_n P_0$. Сумма площадей этих треугольников равна площади многоугольника.

$$A = \sum_{i=1}^n S_i, \quad S_i = \frac{1}{2} * \left| \begin{pmatrix} (P_i - C_0)_x & (P_i - C_0)_y \\ (P_{i+1} - C_0)_x & (P_{i+1} - C_0)_y \end{pmatrix} \right|, \quad i = \{1, n\}$$

A - площадь заданного многоугольника.

3.4.2 Псевдокод алгоритма

Algorithm 8 Нахождение площади многоугольника

```
1: function GETAREA(Polygon2D)
2:   for  $i = 0$  to  $n$  do            $\triangleright n$  - количество вершин в многоугольнике
3:      $c_0 \leftarrow c_0 + \text{Polygon2D.Point}_i$ 
4:   end for
5:    $c_0 \leftarrow c_0 / n$ 
6:   for  $i = 0$  to  $n$  do
7:      $p_1 \leftarrow \text{Polygon2D.Point}_i$ 
8:     if  $i + 1 \neq n$  then
9:        $p_2 \leftarrow \text{Polygon2D.Point}_{i+1}$ 
10:    else
11:       $p_2 \leftarrow \text{Polygon2D.Point}_0$ 
12:    end if
13:     $a \leftarrow p_1 - c_0$ 
14:     $b \leftarrow p_2 - c_0$ 
15:     $s \leftarrow \text{VectorProduct2D}(a, b)$             $\triangleright s$  - Площадь  $\triangle p_1 p_2 c_0$ 
16:     $\text{Result} \leftarrow \text{Result} + s$ 
17:  end for
18: end function
```

Данная функция вызывается как метод класса Polygon2D

3.5 Нахождение сечения заданной площади, заданным направлением сечения

Уравнение плоскости в пространстве:

$$L : Ax + By + Cz + D = 0$$

A, B, C - это направление вектора нормали плоскости, а D - положение плоскости в пространстве. Требуется найти такое значение D , чтобы площадь сечения, построенного плоскостью L , была равна заданной площади.

3.5.1 Алгоритм

Используем метод деления отрезка пополам. Началом и концом отрезка будут такие значения D , что площадь сечения, построенного плоскостью L , минимальна и максимальна соответственно.

Площадь сечения будет минимальным, если плоскость сечения - это касательная к многограннику.

Площадь сечения будет максимальным, если плоскость сечения проходит через центр многогранника. Найдем такое значение D , при котором плоскость L проходит через центр многогранника. Это значение будет концом отрезка.

Методом деления отрезка пополам найдем значение D , при котором грань сечения плоскостью L будет нужной площади.

3.5.2 Псевдокод алгоритма

Algorithm 9 Центр многогранника

```
1: function CENTEROFPOLYHEDRON(Polyhedron)
2:    $a = \text{Projection}(\text{Polyhedron}, Ox)$ 
3:    $b = \text{Projection}(\text{Polyhedron}, Oy)$ 
4:    $c = \text{Projection}(\text{Polyhedron}, Oz)$ 
5:    $p_1 = \text{Middle}(a)$ 
6:    $p_2 = \text{Middle}(b)$ 
7:    $p_3 = \text{Middle}(c)$ 
8:    $\text{Result}(p_1, p_2, p_3)$ 
9:   return  $\text{Result}$ 
10: end function
```

Algorithm 10 Сечение максимальной площади

```
1: function MAXAREA(Polyhedron, SectionPlane)
2:    $ResPoint \leftarrow CENTEROFPOLYHEDRON(Polyhedron)$ 
3:    $Result \leftarrow SectionPlane.A * ResPoint.X + SectionPlane.B * ResPoint.Y + SectionPlane.C * ResPoint.Z$ 
4:    $Result \leftarrow -Result$ 
5:   return  $Result$ 
6: end function
```

Algorithm 11 Сечение минимальной площади

```
1: function MINAREA(Polyhedron, SectionPlane)
2:    $SectionPlane.setD(MaxNumber)$   $\triangleright$  Максимально удаляем  $SectionPlane$  от  $Polyhedron$ , чтобы они не пересекались
3:    $MinDistance \leftarrow MaxNumber$ 
4:   for  $i = 0$  to  $n$  do  $\triangleright n$ -Количество вершин в многограннике
5:     if  $Distance(SectionPlane, Polyhedron.Point_i) < MinDistance$  then
6:        $MinDistance \leftarrow Distance(SectionPlane, Polyhedron.Point_i)$ 
7:        $ResPoint \leftarrow Polyhedron.Point_i$ 
8:     end if
9:      $Result \leftarrow SectionPlane.A * ResPoint.X + SectionPlane.B * ResPoint.Y + SectionPlane.C * ResPoint.Z$ 
10:     $Result \leftarrow -Result$ 
11:    return  $Result$ 
12:  end for
13: end function
```

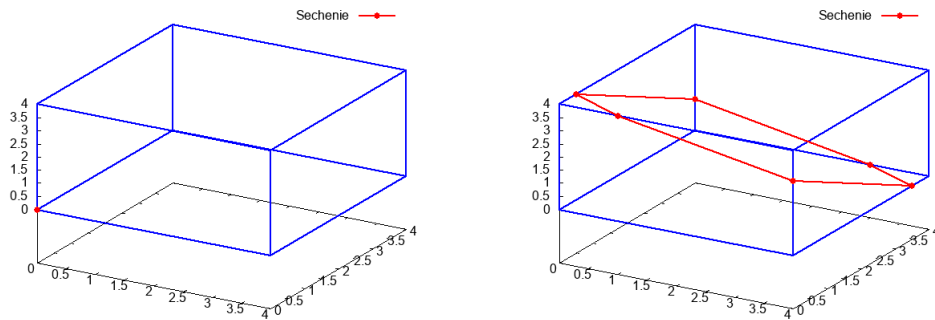
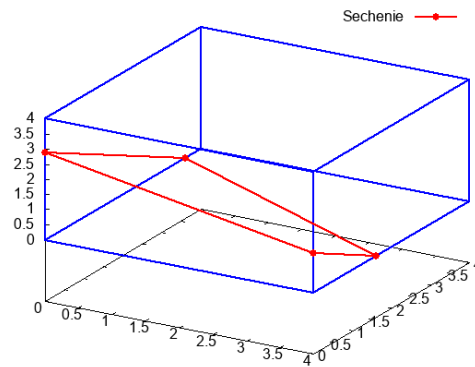


Рис. Сечения минимальной(0.0) и максимальной площади(20.7954), направления сечения (1,2,2.5).

Algorithm 12 Сечение заданной площади

```
1: function SAMEAREA(Polyhedron, SectionPlane, Area)
2:    $Start \leftarrow \text{MINAREA}(Polyhedron, SectionPlane)$ 
3:    $End \leftarrow \text{MAXAREA}(Polyhedron, SectionPlane)$ 
4:    $SectionPlane.SetD(Start)$ 
5:   if  $Area \leq \text{SECTION2D}(Polyhedron, SectionPlane).GetArea()$  then
6:     return  $Start$ 
7:   end if
8:    $SectionPlane.SetD(End)$ 
9:   if  $Area \geq \text{SECTION2D}(Polyhedron, SectionPlane).GetArea()$  then
10:    return  $End$ 
11:  end if
12:  while true do
13:     $Result \leftarrow (Start + End)/2.0$ 
14:     $SectionPlane.SetD(Result)$ 
15:     $ResArea \leftarrow \text{SECTION2D}(Polyhedron, SectionPlane).GetArea()$ 
16:    if  $ResArea \leq Area$  then
17:       $Start \leftarrow (Start + End)/2.0$ 
18:    else if  $ResArea \geq Area$  then
19:       $End \leftarrow (Start + End)/2.0$ 
20:    end if
21:    if  $|Area - ResArea| \leq ERROR$  then break
22:    end if
23:  end while
24:  return  $Result$ 
25: end function
```



Сечение площади равной, например, 14

3.6 Вычисление нормы разности многоугольников

Полярная сетка - это множество лучей с общей начальной точкой и с угловым шагом $h = 2\pi/N$ между ними, где N - количество узлов полярной сетки.

3.6.1 Алгоритм

Перенесем многоугольник на полярную сетку, т.е. построим полярную сетку, началом которой будет центр тяжести многоугольника. Лучи как-то пересекут стороны многоугольника. Получили набор радиус-векторов R_j . Начало радиус-вектора R_j - это начало полярной сетки, а конец - точка пересечения j -го луча и стороны многоугольника.

Второй многоугольник тоже перенесем на полярную сетку и получим набор радиус-векторов F_j .

Вычислим:

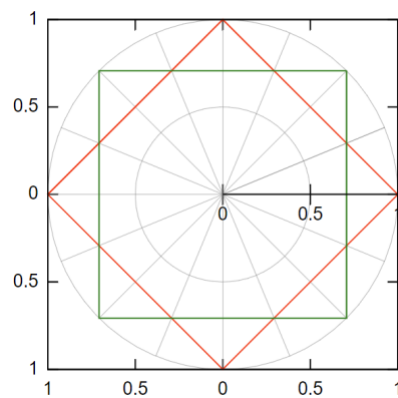
$$D = \sqrt{\sum_{j=1}^N (R_j - F_j)^2}$$

Но в исходном положении многоугольники могут оказаться неблизкими, поэтому мы поворачиваем один из многоугольников на полярной сетке. Это соответствует циклическому сдвигу массива R_j , т.е:

$$D_s = \sqrt{\sum_{j=1}^N (R_{j+s} - F_j)^2}, \quad s = \{0, 1, 2, \dots, N-1\}$$

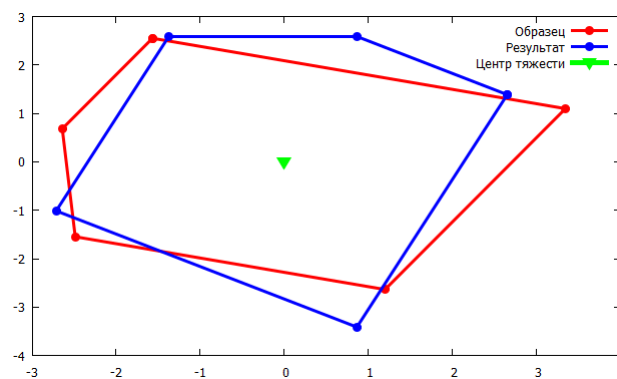
$$D_k = \text{Min}(D_s), \quad \varphi = k * \frac{2\pi}{N}$$

D_k - норма разности многоугольников, φ - угол поворота, при котором многоугольники наиболее близки.

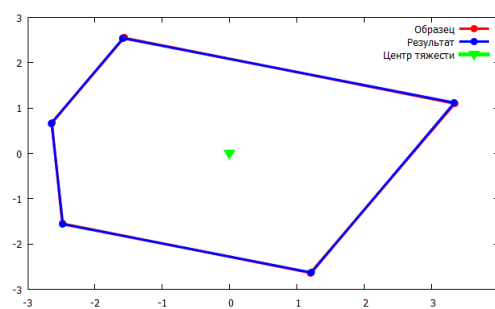
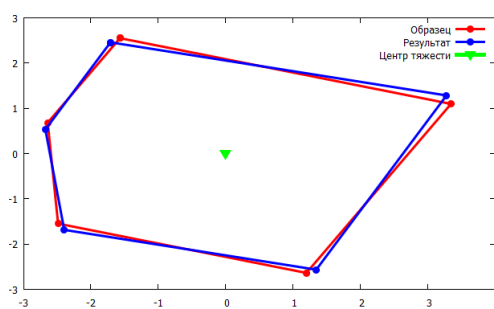


Пример поворота на величину $s = 2$ на сетке с количеством узлов $N = 16$

Результаты работы алгоритма:



Многоугольники до поворота на найденную величину



Многоугольники после поворота

- 1) Количество узлов сетки = 100,
- 2) Количество узлов сетки = 1000

3.6.2 Псевдокод алгоритма

Построение полярной сетки

Algorithm 13 Построение полярной сетки

```
1: function POLARGRID(Count, Length)
2:   for  $i = 0$  to Count do
3:      $a \leftarrow \text{Point2D}(\text{Length} * \cos(\frac{2\pi*i}{\text{Count}}), \text{Length} * \sin(\frac{2\pi*i}{\text{Count}}))$ 
4:      $res \leftarrow \text{Segment}(O, a)$  ▷  $O$ -Начало координат
5:      $\text{Result.Push\_Back}(res)$ 
6:   end for
7:   return Result
8: end function
```

Представление многоугольника на полярной сетке

Algorithm 14 Многоугольник на полярной сетке

```
1: function POLYGONINPOLAR(Polygon2D, Count)
2:    $\text{Polygon2D.MoveToCenter}()$ 
3:    $\text{Diameter} \leftarrow \text{Polygon2D.GetDiameter}()$ 
4:    $\text{Ray} \leftarrow \text{POLARGRID}(\text{Count}, \text{Diameter})$ 
5:   for  $i = 0$  to Count do
6:     for  $j = 0$  to  $n$  do ▷  $n$ -Количество сторон многоугольника
7:       if ARE_INTERSECT( $\text{Ray}_i, \text{Polygon2D.Side}_j$ ) then
8:          $x \leftarrow \text{INTERSECT}(\text{Ray}_i, \text{Polygon2D.Side}_j)$ 
9:          $\text{Result.Push\_Back}(\text{Distance}(O, x))$  ▷  $O$ -начало
координат
10:      end if
11:    end for
12:  end for
13:  return Result
14: end function
```

Получили список радиус-векторов

Вычисление нормы разности многоугольников на полярной сетке

Algorithm 15 Норма разности многоугольников

```

1: function NORMARAZNOSTI(Polygon2D1, Polygon2D2, Count)
2:   res  $\leftarrow$  0.0
3:   Norma  $\leftarrow$  MaxNumber
4:   Diameter1  $\leftarrow$  Polygon2D1.GetDiameter()
5:   Diameter2  $\leftarrow$  Polygon2D2.GetDiameter()
6:   Polygon1  $\leftarrow$  POLYGONINPOLAR(Polygon2D1, Count)
7:   Polygon2  $\leftarrow$  POLYGONINPOLAR(Polygon2D2, Count)  $\triangleright$  Polygon1 и
   Polygon2 - списки радиус-векторов Polygon2D1 и Polygon2D2 соот-
   ветственно
8:   for k = 0 to Count do
9:     i  $\leftarrow$  i + k  $\triangleright$  Циклический сдвиг радиус-векторов Polygon1
10:    for j = 0 to Count do
11:      s  $\leftarrow$  (Polygon1,i - Polygon2,j)
12:      s  $\leftarrow$  s * s
13:      res  $\leftarrow$  res + s
14:      i  $\leftarrow$  i + 1
15:    end for
16:    res  $\leftarrow$   $\sqrt[2]{res}$ 
17:    if res < Norma then
18:      Norma  $\leftarrow$  res
19:      BestAngle  $\leftarrow$   $\frac{2\pi}{Count} * k$ 
20:      res  $\leftarrow$  0.0
21:    end if
22:  end for
23:  return Norma, BestAngle
24: end function

```

3.7 Аналитическое вычисление глубины сечения

Пусть уже найдена некоторая грань сечения, но пока она не подходит по площади. Зафиксируем направление сечения. При изменении глубины сечения, вершины грани сечения скользят по ребрам. Выразим эти вершины как линейные функции, зависящие от D , т. е. от глубины сечения. Все вершины грани сечения - это пересечение ребер, отходящих от вершин уже найденной грани, и плоскости сечения.

Пусть у найденной грани сечения n вершин. Соответственно у нас будет n ребер R_i . Прямая на которой лежит R_i :

$$\frac{x_i - x_{i0}}{m_i} = \frac{y_i - y_{i0}}{n_i} = \frac{z_i - z_{i0}}{p_i}$$

Плоскость сечения: $Ax + By + Cz + D = 0$, A, B, C -зафиксированы.

Для нахождения вершины P_i , решим систему:

$$\begin{cases} \frac{x_i - x_{i0}}{m_i} = \frac{y_i - y_{i0}}{n_i} = \frac{z_i - z_{i0}}{p_i} \\ Ax_i + By_i + Cz_i + D = 0 \end{cases}$$

$$\begin{cases} x_i = \lambda_i m_i + x_{i0} \\ y_i = \lambda_i n_i + y_{i0} \\ z_i = \lambda_i p_i + z_{i0} \\ Ax_i + By_i + Cz_i + D = 0 \end{cases}$$

$\Downarrow$

$$A(\lambda_i m_i + x_{i0}) + B(\lambda_i n_i + y_{i0}) + C(\lambda_i p_i + z_{i0}) + D = 0$$

$$\lambda_i = -\frac{Ax_{i0} + By_{i0} + Cz_{i0}}{Am_i + Bn_i + Cp_i} - \frac{1}{Am_i + Bn_i + Cp_i} D$$

$$\lambda_i = \alpha_i + \beta_i D$$

$\Downarrow$

$$\begin{cases} x_i = (\alpha_i + \beta_i D)m_i + x_{i0} = (\alpha_i m_i + x_{i0}) + (m_i \beta_i)D = \gamma_{xi} + \delta_{xi} D \\ y_i = (\alpha_i + \beta_i D)n_i + y_{i0} = (\alpha_i n_i + y_{i0}) + (n_i \beta_i)D = \gamma_{yi} + \delta_{yi} D \\ z_i = (\alpha_i + \beta_i D)p_i + z_{i0} = (\alpha_i p_i + z_{i0}) + (p_i \beta_i)D = \gamma_{zi} + \delta_{zi} D \end{cases}$$

$$P_i(D) = \begin{pmatrix} \gamma_{xi} + \delta_{xi} D \\ \gamma_{yi} + \delta_{yi} D \\ \gamma_{zi} + \delta_{zi} D \end{pmatrix}, \quad i = \{\overline{0, n-1}\}$$

Нашли вершины P_i . Теперь, чтобы найти площадь, разобьем грань сечения на треугольники вида $\triangle P_0 P_i P_{i+1}$. Сумма площадей этих треугольников равна площади грани сечения. Площадь треугольника $\triangle_i$ равна половине длины вектора v_i , который получается из векторного произведения векторов $\vartheta_i = P_0 - P_i$ и $\vartheta_{i+1} = P_0 - P_{i+1}$.

$$\vartheta_i = P_0 - P_i, \quad i = \{\overline{1, n-1}\}$$

$$v_i = \vartheta_i \times \vartheta_{i+1} = \begin{pmatrix} \zeta_{xi} + \eta_{xi}D + \theta_{xi}D^2 \\ \zeta_{yi} + \eta_{yi}D + \theta_{yi}D^2 \\ \zeta_{zi} + \eta_{zi}D + \theta_{zi}D^2 \end{pmatrix}, \quad i = \{\overline{1, n-2}\}$$

Все векторы ϑ_i лежат на одной плоскости. Поэтому длина вектора v_i - это скалярное произведение этого вектора и единичной нормали плоскости, на которой лежат векторы ϑ_i .

Единичный вектор нормали плоскости сечения:

$$\tau = \left(\frac{A}{\sqrt{A^2 + B^2 + C^2}}, \frac{B}{\sqrt{A^2 + B^2 + C^2}}, \frac{C}{\sqrt{A^2 + B^2 + C^2}} \right)$$

$$\rho_i = \frac{v_i * \tau}{2} = \zeta_i + \phi_i D + \psi_i D^2, \quad i = \{\overline{1, n-2}\}$$

ρ_i -площадь треугольника $\triangle_i$. Просуммируем их и получим площадь грани сечения.

$$Area = \sum_{i=1}^{n-2} \rho_i = \Phi + \Psi D + \Omega D^2$$

Мы свели нахождение сечения заданной площади к решению квадратного уравнения. Решим это квадратное уравнение и получим нужную глубину сечения.

$$\begin{aligned} \Phi' &= \Phi - Area \\ \Omega D^2 + \Psi D + \Phi' &= 0 \\ D_{1,2} &= \frac{-\Psi \pm \sqrt{\Psi^2 - 4\Omega\Phi'}}{2\Omega} \end{aligned}$$

Из двух решений выберем то, которое лежит в окрестности уже найденной грани.

Но так как мы использовали уравнения линий ребер, может случиться так, что на найденной глубине плоскость сечения уже не пересекает само ребро. В этом случае необходимо использовать поиск глубины методом отрезком пополам(3.5). Для проверки этого случая, достаточно найти все $P_i(D)$, и проверить, лежит ли точка P_k на ребре R_k , $k = \{\overline{1, n}\}$.

3.8 Поиск направления сечения шагами по двумерной сетке

3.8.1 Алгоритм

Пусть у нас есть какое-то начальное направление и мы нашли сечение по глубине, площадь которого равна площади образца. Так как направление не может сразу оказаться правильным, между найденным сечением и образцом есть расхождение. Теперь надо выбрать другое направление, чтобы полученное по этому направлению сечение лучше соответствовало образцу. Для этого возьмем текущее направление и отклонить его в разные стороны с некоторым шагом. Количество таких проб - 8 - это влево, вправо, вверх, вниз и еще 4 промежуточных. Потом выбираем направление, при котором совпадение сечения с образцом (в смысле максимального диаметра и габаритов) окажется лучше, и отправляясь от него также отклонения уже от этого направления. Так как мы наклоняем направление плоскости сечения на маленькие шаги, новая грань сечения, в большинстве случаев, останется на тех же ребрах, которые отходят от вершин предыдущей грани сечения. Поэтому там, где новое сечение не выходит за пределы ребер, отходящих от вершин текущего сечения, будем использовать аналитический поиск глубины сечения и построим сечение как пересечение текущих ребер и плоскости сечения. Если отклонения не дают более хорошего приближения, уменьшим шаг отклонения в два раза. Алгоритм останавливается, когда шаг будет меньше требуемой точности.

3.8.2 Псевдокод алгоритма

Algorithm 16 Ребра, отходящие от вершин грани сечения

```
1: function GETEDGES(Polyhedron, Polygon)
2:   for  $i = 0$  to  $n$  do                                ▷ n-Количество вершин Polygon
3:     for  $j = 0$  to  $k$  do                                ▷ k-Количество ребер Polyhedron
4:       if PointInRebro(Polygon.Point[ $i$ ], Polyhedron.Edge[ $j$ ]) then
5:         Result.PUSH_BACK(Polyhedron.Edge[ $j$ ])
6:       end if
7:     end for
8:   end for
9:   return Result
10: end function
```

Algorithm 17 Поиск наилучшего направления в смысле диаметра

```
1: function SAMEDIAMETER(Polyhedron, SectionPlane, Obrazec )
2:   step  $\leftarrow$  1
3:   area  $\leftarrow$  Obrazec.GETAREA( )
4:   diameter  $\leftarrow$  Obrazec.GETDIAMETER( )
5:   SectionPlane.SETD(SameArea(Polyhedron, SectionPlane, area))
6:   resPolygon3D  $\leftarrow$  SECTION3D(Polyhedron, SectionPlane)
7:   resPolygon  $\leftarrow$  GETPOLYGON2D(resPolygon3D)
8:   resDiameter  $\leftarrow$  resPolygon.GETDIAMETER( )
9:   raznica  $\leftarrow$  |diameter – resDiameter|
10:  ListOfEdges  $\leftarrow$  GETEDGES(Polyhedron, resPolygon3D)
11:  while raznica  $\geq$  TOCHNOST do
12:    TestPlane  $\leftarrow$  SectionPlane
13:    TestPlane.NAKLON(step)
14:    if ANALITICD(ListOfEdges, TestPlane, area).Exist then
15:      TestPlane.SETD(AnaliticD(ListOfEdges, TestPlane, area))
16:    else
17:      TestPlane.SETD(SameArea(Polyhedron, SectionPlane, area))
18:    end if
19:    if LITESECTION(ListOfEdges, TestPlane).Exist then
20:      resPolygon3D  $\leftarrow$  LITESECTION(ListOfEdges, TestPlane)
21:      resPolygon  $\leftarrow$  GETPOLYGON2D(resPolygon3D)
22:    else
23:      resPolygon3D  $\leftarrow$  SECTION3D(Polyhedron, SectionPlane)
24:      resPolygon  $\leftarrow$  GETPOLYGON2D(resPolygon3D)
25:      ListOfEdges  $\leftarrow$  GETEDGES(Polyhedron, resPolygon3D)
26:    end if
27:    resDiameter  $\leftarrow$  resPolygon.GETDIAMETER( )
28:    if |diameter – resDiameter| < raznica then
29:      SectionPlane  $\leftarrow$  TestPlane
30:      raznica  $\leftarrow$  |diameter – resDiameter|
31:    else
32:      step  $\leftarrow$  step/2.0
33:      if step < TOCHNOSTSTEP then
34:        Break
35:      end if
36:    end if
37:  end while
38:  return SectionPlane
39: end function
```

Algorithm 18 Поиск наилучшего направления в смысле габаритов

```
1: function SAME_SIZE(Polyhedron, SectionPlane, Obrazec )
2:   step  $\leftarrow$  1
3:   area  $\leftarrow$  Obrazec.GETAREA( )
4:   size  $\leftarrow$  Obrazec.GETSIZE( )
5:   SectionPlane.SETD(SameArea(Polyhedron, SectionPlane, area))
6:   resPolygon3D  $\leftarrow$  SECTION3D(Polyhedron, SectionPlane)
7:   resPolygon  $\leftarrow$  GETPOLYGON2D(resPolygon3D)
8:   resSize  $\leftarrow$  resPolygon.GETSIZE( )
9:   raznica  $\leftarrow$  |size – resSize|
10:  ListOfEdges  $\leftarrow$  GETEDGES(Polyhedron, resPolygon3D)
11:  while raznica  $\geq$  TOCHNOST do
12:    TestPlane  $\leftarrow$  SectionPlane
13:    TestPlane.NAKLON(step)
14:    if ANALITICD(ListOfEdges, TestPlane, area).Exist then
15:      TestPlane.SETD(AnaliticD(ListOfEdges, TestPlane, area))
16:    else
17:      TestPlane.SETD(SameArea(Polyhedron, SectionPlane, area))
18:    end if
19:    if LITESECTION(ListOfEdges, TestPlane).Exist then
20:      resPolygon3D  $\leftarrow$  LITESECTION(ListOfEdges, TestPlane)
21:      resPolygon  $\leftarrow$  GETPOLYGON2D(resPolygon3D)
22:    else
23:      resPolygon3D  $\leftarrow$  SECTION3D(Polyhedron, SectionPlane)
24:      resPolygon  $\leftarrow$  GETPOLYGON2D(resPolygon3D)
25:      ListOfEdges  $\leftarrow$  GETEDGES(Polyhedron, resPolygon3D)
26:    end if
27:    resSize  $\leftarrow$  resPolygon.GETSIZE( )
28:    if |size – resSize| < raznica then
29:      SectionPlane  $\leftarrow$  TestPlane
30:      raznica  $\leftarrow$  |size – resSize|
31:    else
32:      step  $\leftarrow$  step/2.0
33:      if step < TOCHNOSTSTEP then
34:        Break
35:      end if
36:    end if
37:  end while
38:  return SectionPlane
39: end function
```

Algorithm 19 Поиск наилучшего направления в смысле нормы разности

```
1: function BESTNORMA(Polyhedron, SectionPlane, Obrazec )
2:   step  $\leftarrow$  1
3:   area  $\leftarrow$  Obrazec.GETAREA( )
4:   SectionPlane.SETD(SameArea(Polyhedron, SectionPlane, area))
5:   resPolygon3D  $\leftarrow$  SECTION3D(Polyhedron, SectionPlane)
6:   resPolygon  $\leftarrow$  GETPOLYGON2D(resPolygon3D)
7:   norma  $\leftarrow$  NORMARAZNOSTI(Obrazec, resPolygon, N)
8:   ListOfEdges  $\leftarrow$  GETEDGES(Polyhedron, resPolygon3D)
9:   while norma  $\geq$  TOCHNOST do
10:    TestPlane  $\leftarrow$  SectionPlane
11:    TestPlane.NAKLON(step)
12:    if ANALITICD(ListOfEdges, TestPlane, area).Exist then
13:      TestPlane.SETD(AnaliticD(ListOfEdges, TestPlane, area))
14:    else
15:      TestPlane.SETD(SameArea(Polyhedron, SectionPlane, area))
16:    end if
17:    if LITESECTION(ListOfEdges, TestPlane).Exist then
18:      resPolygon3D  $\leftarrow$  LITESECTION(ListOfEdges, TestPlane)
19:      resPolygon  $\leftarrow$  GETPOLYGON2D(resPolygon3D)
20:    else
21:      resPolygon3D  $\leftarrow$  SECTION3D(Polyhedron, SectionPlane)
22:      resPolygon  $\leftarrow$  GETPOLYGON2D(resPolygon3D)
23:      ListOfEdges  $\leftarrow$  GETEDGES(Polyhedron, resPolygon3D)
24:    end if
25:    resNorma  $\leftarrow$  NORMARAZNOSTI(Obrazec, resPolygon, N)
26:    if resNorma < norma then
27:      SectionPlane  $\leftarrow$  TestPlane
28:      norma  $\leftarrow$  resNorma
29:    else
30:      step  $\leftarrow$  step/2.0
31:      if step < TOCHNOSTSTEP then
32:        Break
33:      end if
34:    end if
35:  end while
36:  return SectionPlane
37: end function
```

Algorithm 20 Поиск по диаметру и габаритам

```
1: function SEARCH(Polyhedron, SectionPlane, Образец)
2:   SectionPlane  $\leftarrow$  (1, 1, 1, -1)
3:   SectionPlane  $\leftarrow$  SAMEDIAMETER(Polyhedron, SectionPlane, Образец)
4:   SectionPlane  $\leftarrow$  SAME SIZE(Polyhedron, SectionPlane, Образец)
5:   SectionPlane  $\leftarrow$  BESTNORMA(Polyhedron, SectionPlane, Образец)
6:   return SectionPlane
7: end function
```

3.9 Аналитический метод решения задачи (не реализован)

Пусть у нас есть некоторое сечение. После того, как перенесли сечение на плоскость и нашли наилучшее совмещение сечения и образца, можем оценить насколько один многоугольник отличается от второго в фиксированной системе координат. Теперь, исходя от различий многоугольников, наклоняем нормаль плоскости сечения. Но тут мы будем интерпретировать так, что плоскость сечения остается неподвижным, а поворачивается весь многогранник.

Итак, есть некоторое сечение. Будем считать, что плоскость этого сечения описывается уравнением $z = 0$. В этой плоскости лежит грань сечения и тестовый многоугольник, и центры этих многоугольников находятся в начале координат. Соответственно ребра сечения и тестового многоугольника описываются какими-то прямыми типа $A_i x + B_i y + D_i = 0$ в плоскости $z = 0$, а смежные грани многоугольника описываются какими-то плоскостями $A_j x + B_j y + C_j z + D_j = 0$. Теперь наклоняем нормаль плоскости сечения. Для того чтобы новая плоскость сечения была равна плоскости $z = 0$, нам нужно повернуть многогранник сначала вокруг оси z , и потом вокруг оси x . Матрица поворота:

Новая нормаль плоскости сечения: (x', y', z')

Угол между новой нормалью и плоскостью $y = 0$:

$$\varphi = \arccos \frac{y'}{\sqrt{x'^2 + y'^2}}$$

Матрица поворота вокруг оси z :

$$M_z(\varphi) = \begin{pmatrix} \cos \varphi & -\sin \varphi & 0 \\ \sin \varphi & \cos \varphi & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} \frac{y'}{\sqrt{x'^2 + y'^2}} & -\frac{x'}{\sqrt{x'^2 + y'^2}} & 0 \\ \frac{x'}{\sqrt{x'^2 + y'^2}} & \frac{y'}{\sqrt{x'^2 + y'^2}} & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Угол между новой нормалью и нормалью плоскости $z = 0$:

$$\psi = \arccos \frac{z'}{\sqrt{y'^2 + z'^2}}$$

Матрица поворота вокруг оси x :

$$M_x(\psi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \psi & -\sin \psi \\ 0 & \sin \psi & \cos \psi \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \frac{z'}{\sqrt{y'^2 + z'^2}} & -\frac{y'}{\sqrt{y'^2 + z'^2}} \\ 0 & \frac{y'}{\sqrt{y'^2 + z'^2}} & \frac{z'}{\sqrt{y'^2 + z'^2}} \end{pmatrix}$$

Вся матрица поворота:

$$M = M_z(\varphi) \times M_x(\psi) = \begin{pmatrix} \frac{y'}{\sqrt{x'^2 + y'^2}} & -\frac{x'z'}{\sqrt{x'^2 + y'^2}\sqrt{y'^2 + z'^2}} & \frac{x'y'}{\sqrt{x'^2 + y'^2}\sqrt{y'^2 + z'^2}} \\ \frac{x'}{\sqrt{x'^2 + y'^2}} & \frac{y'z'}{\sqrt{x'^2 + y'^2}\sqrt{y'^2 + z'^2}} & -\frac{y'^2}{\sqrt{x'^2 + y'^2}\sqrt{y'^2 + z'^2}} \\ 0 & \frac{y'}{\sqrt{y'^2 + z'^2}} & \frac{z'}{\sqrt{y'^2 + z'^2}} \end{pmatrix}$$

Грани многогранника после поворота:

$$F'_i = F_i \times M, i = \{\overline{1, n}\}$$

Вершины многогранника после поворота:

$$P'_j = P_j \times M, j = \{\overline{1, k}\}$$

Получили новый многогранник:

$$I \mapsto I'$$

Теперь предлагается найти грань сечения I' с плоскостью $z = 0$, перенести грань сечения на полярную сетку и найти такие значения (x', y', z') , что расстояние между сечением и образцом будет минимальной.

Глава 4

Заключение

4.1 Выводы

- 1 Был реализован алгоритм пересечения многогранника плоскостью. Тестирование показало, что алгоритм работает в самых разнообразных случаях.
- 2 Был разработан алгоритмы совмещения многоугольников и вычисления расстояния между плоскими многоугольниками. Алгоритмы были воспроизведены и протестированы в ряде случаев и дали положительный результат, показав свою работоспособность.
- 3 Были разработаны алгоритмы нахождения сечения заданной площади с заданной нормалью двумя методами: итерационным и аналитическим. Оба метода были воспроизведены и протестированы и дали положительный результат, показав свою работоспособность.
- 4 Был разработан алгоритм поиска направления для сечения с наилучшим совпадением с образцом, а также разработанная её программная реализация, тестирование которой дала положительный результат. Наборы тестовых примеров представлены в следующем разделе.
- 5 Задача была полностью решена итерационным методом для выпуклых многогранников.
- 6 Нерешенной проблемой осталось решение задачи аналитическим методом. Для этой проблемы был предложен подход в разделе (3.9). Реализация этого метода представляет собой весьма интересную задачу.

4.2 Результаты

В программу в качестве многогранника задан куб 4x4x4. Допустимая норма разности: 0,01.

Функция *main* программы:

```
1  int main()
2  {
3      Point a[8];
4      a[0].setxyz(0,0,0);
5      a[1].setxyz(4,0,0);
6      a[2].setxyz(4,4,0);
7      a[3].setxyz(0,4,0);
8      a[4].setxyz(0,4,4);
9      a[5].setxyz(0,0,4);
10     a[6].setxyz(4,0,4);
11     a[7].setxyz(4,4,4);
12     \\Vershini
13     Plane B[6];
14     B[0].set(a[0],a[1],a[5]);
15     B[1].set(a[7],a[3],a[2]);
16     B[2].set(a[0],a[1],a[2]);
17     B[3].set(a[1],a[2],a[6]);
18     B[4].set(a[4],a[6],a[7]);
19     B[5].set(a[0],a[3],a[4]);
20     \\Uравneniya ploskostey po trem tochkam
21     list<int> c[6];
22     c[0].push_back(0);
23     c[0].push_back(1);
24     c[0].push_back(6);
25     c[0].push_back(5);
26     c[1].push_back(3);
27     c[1].push_back(2);
28     c[1].push_back(7);
29     c[1].push_back(4);
30     c[2].push_back(0);
31     c[2].push_back(1);
32     c[2].push_back(2);
33     c[2].push_back(3);
34     c[3].push_back(1);
```

```

35     c[3].push_back(2);
36     c[3].push_back(7);
37     c[3].push_back(6);
38     c[4].push_back(4);
39     c[4].push_back(5);
40     c[4].push_back(6);
41     c[4].push_back(7);
42     c[5].push_back(0);
43     c[5].push_back(3);
44     c[5].push_back(4);
45     c[5].push_back(5);
46     \\Spisok numerov vershin
47     Gran G[6];
48     G[0].setgran(B[0],c[0]);
49     G[1].setgran(B[1],c[1]);
50     G[2].setgran(B[2],c[2]);
51     G[3].setgran(B[3],c[3]);
52     G[4].setgran(B[4],c[4]);
53     G[5].setgran(B[5],c[5]);
54     \\Zadali grani po uravneniyu ploskosti i spisku numerov vershin
55     Polyhedron Cube(8,a,6,G);
56     \\Zadali mnogogrannik "Cube"
57     Plane obr(9.61,8.64,3.48,-40.2);
58     Polygon2D obrazec=SECHENIE2D(Cube, obr);
59     obrazec.rotate(1.52);
60     \\V kachestve obrazca vzyali nekotrooe sechenie kuba i povernuli
61
62     Plane test;
63     test=SEARCH(Cube,obrazec);
64     \\Primenili algoritm
65     setlocale(LC_ALL, "Russian");
66     Polygon2D poly=SECHENIE2D(Cube,test);
67     \\Postroili sechenie po najdennomu uravneniyu ploskosti
68     RAZNOST norma=Norma_raznosti_polygon(obrazec,poly,1024);
69     \\Vichislili normu raznosti
70     poly.rotate(norma.angle);
71     \\Povernuli najdenney mnogougolnik
72     system("cls");
73     cout<<"Diametr_Obrazca:_ "<<obrazec.getMaxRadius()<<endl;
74     cout<<"Ploshad_obrazca:_ "<<obrazec.getArea()<<endl;
75     cout<<"===== "<<endl;

```

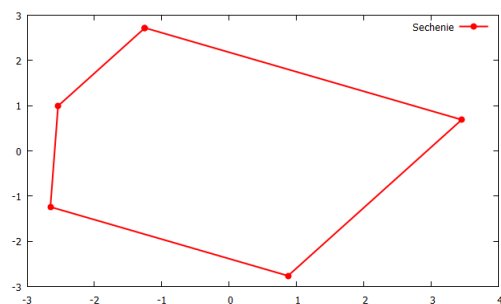
```

76     cout<<"Diametr_najdennogo_secheniya:_ "<<poly.getMaxRadius()<<endl;
77     cout<<"Ploshad_najdennogo_secheniya:_ "<<poly.getArea()<<endl;
78     cout<<"Napravlenije_secheniya:_ ("<<test.getA()<<" ,_"<<test.getB()<<endl;
79     cout<<"Ugol_povorota_secheniya:_ "<<norma.angle<<endl;
80     cout<<"===== "<<endl;
81     cout<<"Norma_raznosti_secheniya_i_obrazca:_ "<<norma.norma<<endl;
82     cout<<"Kolishectvo_uzlov_polyarnoj_setki:_ _1024"<<endl;
83     obrazec=Sdvig_polygon_to_center( obrazec );
84     poly=Sdvig_polygon_to_center( poly );
85     obrazec.printf(12);
86     poly.printf(13);
87     SECHENIE( Cube, test ,1 );
88     system( "C:/Users/Asus/Documents/Datadiplom/DRAW.bat" );
89     system( "C:/Users/Asus/Documents/Datadiplom/draw2DSechenie_12.plt" )
90 }

```

Тест №1

Образец:

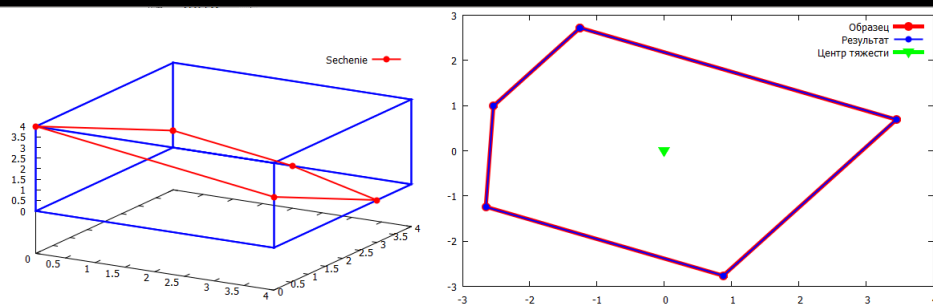


Результат:

```
C:\Users\Asus\... \1_фхх\bin\Debug\1_фххNR.exe
Диаметр образца: 6.40312
Площадь образца: 20.1246
=====
Диаметр найденного сечения: 6.40312
Площадь найденного сечения: 20.1246
Направление сечения: (1, 2.00006, 2.50006), глубина сечения: -10.0002
Угол поворота сечения: 1.52171
=====
Норма разности сечения и образца: 0.00458137
Количество узлов полярной сетки: 1024

C:\Users\Asus\Диплом\1_день>start C:/Users/Asus/Documents/Datadiplom/Animated2DcrossSection/draw2DSechenie_1.plt
C:\Users\Asus\Диплом\1_день>start C:/Users/Asus/Documents/Datadiplom/Animated3DcrossSection/draw3DSechenie_1.plt

Process returned 0 (0x0)   execution time : 103.810 s
Press any key to continue.
```



1) Место сечения в кубе и 2) совмещение сечения с образцом

Процедур было выполнено(Количество):

Сечений: **11235**

Сечений как пересечение отдельных рёбер и плоскости: **544**

Переносов многоугольника на полярную сетку N=100: **96**

Вычислений нормы разности на сетке N=100: **48**

Переносов многоугольника на полярную сетку N=500: **4**

Вычислений нормы разности на сетке N=500: **2**

Переносов многоугольника на полярную сетку N=1024: **2**

Вычислений нормы разности на сетке N=1024: **1**

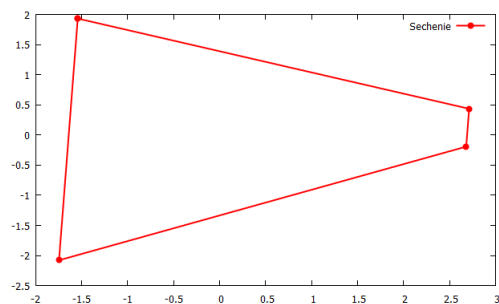
Вычислений глубины сечения итерационным методом: **546**

Вычислений глубины сечения аналитическим методом: **544**

Вычислений габаритов многоугольника: **225**

Тест №2

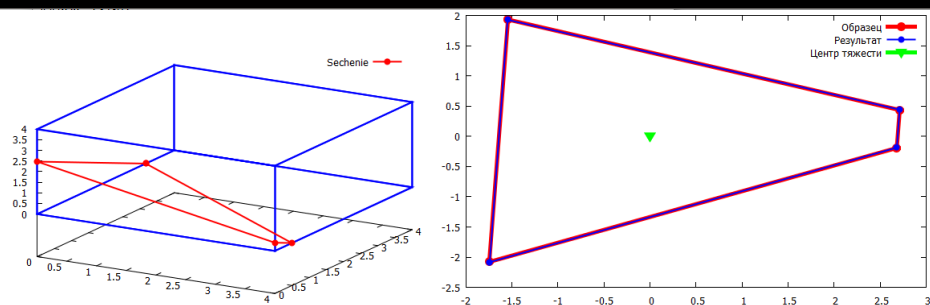
Образец:



Результат:

```
C:\Users\Asus\шяяюк\1_фх№8\bin\Debug\1_фх№8.exe
Диаметр образца: 5.1172
Площадь образца: 10.041
=====
Диаметр найденного сечения: 5.11659
Площадь найденного сечения: 10.041
Направление сечения: (1, 1.49744, 1.9209), глубина сечения: -4.74242
Угол поворота сечения: 3.54656
=====
Норма разности сечения и образца: 0.00502971
Количество узлов полярной сетки: 1024

C:\Users\Asus\Диплом\1_день>start C:/Users/Asus/Documents/Datadiplom/Animated2DcrossSection/draw2DSechenie_1.plt
C:\Users\Asus\Диплом\1_день>start C:/Users/Asus/Documents/Datadiplom/Animated3DcrossSection/draw3DSechenie_1.plt
```



1) Место сечения в кубе и 2) совмещение сечения с образцом

Процедур было выполнено(Количество):

Сечений: **15986**

Сечений как пересечение отдельных рёбер и плоскости: **632**

Переносов многоугольника на полярную сетку N=100: **208**

Вычислений нормы разности на сетке N=100: **104**

Переносов многоугольника на полярную сетку N=500: **4**

Вычислений нормы разности на сетке N=500: **2**

Переносов многоугольника на полярную сетку N=1024: **2**

Вычислений нормы разности на сетке N=1024: **1**

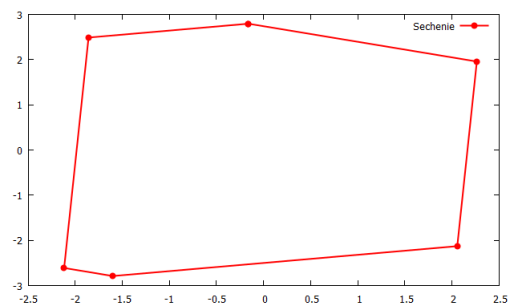
Вычислений глубины сечения итерационным методом: **633**

Вычислений глубины сечения аналитическим методом: **632**

Вычислений габаритов многоугольника: **273**

Тест №3

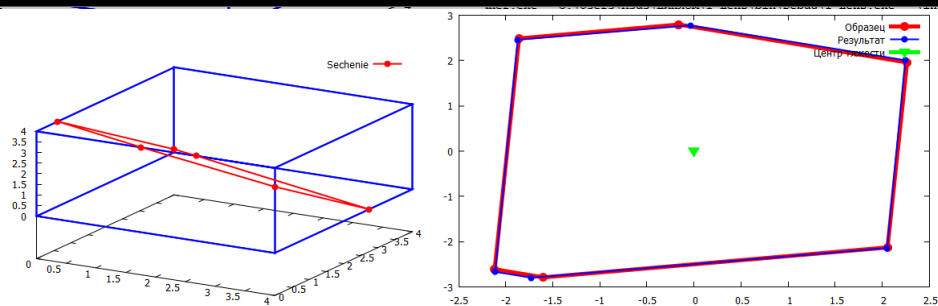
Образец:



Результат:

```
C:\Users\Asus\шляков\1_фх\N8\bin\Debug\1_фх\N8.exe
Диаметр образца: 6.32883
Площадь образца: 20.6228
=====
Диаметр найденного сечения: 6.38837
Площадь найденного сечения: 20.6228
Направление сечения: (1, 2.88269, 2.53699), глубина сечения: -11.9059
Угол поворота сечения: 3.3134
=====
Норма разности сечения и образца: 0.0179489
Количество узлов полярной сетки: 1024

C:\Users\Asus\Диплом\1_день>start C:/Users/Asus/Documents/Datadiplom/Animated2DcrossSection/draw2DSechenie_1.plt
C:\Users\Asus\Диплом\1_день>start C:/Users/Asus/Documents/Datadiplom/Animated3DcrossSection/draw3DSechenie_1.plt
```



1) Место сечения в кубе и 2) совмещение сечения с образцом

Процедур было выполнено(Количество):

Сечений: **8209**

Сечений как пересечение отдельных рёбер и плоскости: **583**

Переносов многоугольника на полярную сетку N=100: **240**

Вычислений нормы разности на сетке N=100: **120**

Переносов многоугольника на полярную сетку N=500: **4**

Вычислений нормы разности на сетке N=500: **2**

Переносов многоугольника на полярную сетку N=1024: **2**

Вычислений нормы разности на сетке N=1024: **1**

Вычислений глубины сечения итерационным методом: **586**

Вычислений глубины сечения аналитическим методом: **584**

Вычислений габаритов многоугольника: **242**

List of Algorithms

1	Пересечение ребра с плоскостью	15
2	Пересечение грани и плоскости	15
3	Алгоритм построения сечения в R_3	16
4	Перевод плоского многоугольника из R_3 в R_2	17
5	Построение сечения на плоскости	17
6	Нахождение центра тяжести многоугольника	18
7	Нахождение габаритов многоугольника	19
8	Нахождение площади многоугольника	20
9	Центр многогранника	21
10	Сечение максимальной площади	22
11	Сечение минимальной площади	22
12	Сечение заданной площади	23
13	Построение полярной сетки	26
14	Многоугольник на полярной сетке	26
15	Норма разности многоугольников	27
16	Ребра, отходящие от вершин грани сечения	30
17	Поиск наилучшего направления в смысле диаметра	31
18	Поиск наилучшего направления в смысле габаритов	32
19	Поиск наилучшего направления в смысле нормы разности	33
20	Поиск по диаметру и габаритам	34

Литература

- [1] *Википедия. Свободная Энциклопедия.* Огранка
(<https://ru.wikipedia.org/wiki/Огранка>)
- [2] Torrence, Bruce Follett. The Student's Introduction to Mathematica .
— Cambridge University Press, 1999. — ISBN 0521594618.
- [3] Murray H. Protter, Charles B. Morrey Jr. College Calculus with
Analytic Geometry. — 2nd. — Reading: Addison-Wesley, 1970.
- [4] Ж. Адамар. Элементарная геометрия(стереометрия). Москва, 1951
- [5] Ю. В. Садовничий, В. В. Федорчук. Аналитическая геометрия.
Москва, 2009